ESMAD | TSIW | PWII
Exercise Sheet nº1 – Introduction to Node.js

Parameters and Environment variables

1. Create a `01_sum.js` script that accepts a set of numbers passed as command line arguments and displays their sum. HINT: use `process.exit(1)` to programmatically kill the Node process with an error code (`1`)

   CALL: node 01_sum.js
   OUTPUT: USAGE: node 01_sum.js <number1> <number2>…<numberN>

   CALL: node 01_sum.js 1a 2 3
   OUTPUT: ERROR: One or more arguments is not a valid number

   CALL: node 01_sum.js 1 2 3 4
   OUTPUT: The sum is 10.

2. Create a `02_welcome.js` script that reads an environment variable called USER_NAME and displays a custom message.

   CALL: node 02_welcome
   OUTPUT: Hello, visitor! Set the USER_NAME variable for a personalized greeting.

   CALL: SET USER_NAME=Teresa && node 02_welcome
   OUTPUT: Hello, Teresa! Welcome to Node!

Node modules

3. Implement a Node module to export the 4 basic mathematical operations between two numbers: addition, subtraction, multiplication, and division. The module must provide an error message if the divider of a division operation is zero.

   Create a Node application that reads the command-line arguments with the format **`operation number1 number2`**, uses the module functions to calculate the result and outputs the result of the operation:

   CALL: node ex3 + 1 2
   OUTPUT: 1 + 2 = 3

   CALL: node ex3 + 1
   OUTPUT: ERROR: Not enough operators! USAGE: node ex3 <operation> <number1> <number2>

   CALL: node ex3 . 1 2
   OUTPUT: ERROR: Invalid operation! Supported operations are: +, -, *, /

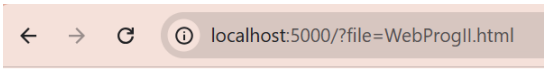
   CALL: node ex3 / 1 0
   OUTPUT: ERROR: Cannot divide by zero!

4. Create an HTTP server with Node.js that recognizes three routes:
    a. GET / → Responds with "Welcome to my server!".
    b. GET /about → Responds with "This is a Node.js server".
    c. GET /time → Responds with the current time (`new Date().toLocaleString()`).

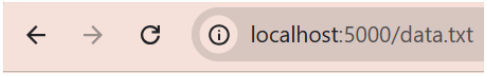
    For all other requests, the server response with “Route not found”.

5. Build a web server accessed through port 5000. It must serve static files inside the `public/` directory, by obtaining the name of the file from the **query string** of the HTTP request.
    If the request corresponds to a **non-existent file**, the server must return a custom HTML page, with error code 404, and list the available files, with links to access them.
    Add some sample files into a `/public` directory inside your application directory and test it.

Welcome!

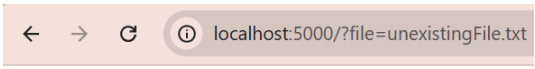


Sample page for WebProgII



Bad Request

key FILE not found in request



404 Not Found

File not found.

Available files:

- data.txt
- ESMAD.png
- WebProgII.html

6. Create a Node.js server that:
    a. When receives a GET request to its root path, serves a custom HTML page, containing a form with an input field to enter a message and a button to send the message via a POST request to route `/message`:

```
<html><body><form action='/message' method='POST'>
<input type='text' name='message' required>
<button type='submit'>Send</button>
</form></body></html>
```

    b. Handle a POST `/message` request by receiving the submitted message and appending it to a `messages.txt` file.
    HINT: https://nodejs.org/en/learn/http/anatomy-of-an-http-transaction#request-body
    After storing the message, respond with status code of 302 (Redirection) and set header “Location” to value (“/”) (this will **redirect the client to the server’s homepage**, and hence displaying the original HTML page with the form).

7. Create the following structure for a small Node “database” management application of users:

```
project
├ users.json // users “database”: start with []
├ app.js     // CLI interface: read the command and arguments, execute the corresponding function from userService, and print the result to the console
└ userService.js    // handle “database” CRUD logic: receives data from main application and performs the necessary file operations
```

The application supports users fundamental CRUD (Create-Read-Update-Delete) operations:

CALL: node app create Ana ana@mail.com
OUTPUT: User created

CALL: node app list
OUTPUT: [{ id: 1, “name”: “Ana”, “email”:”ana@gmail.com”}]

CALL: node app update 1 ana@gmail.com
OUTPUT: User updated

CALL: node app delete 1
OUTPUT: User deleted

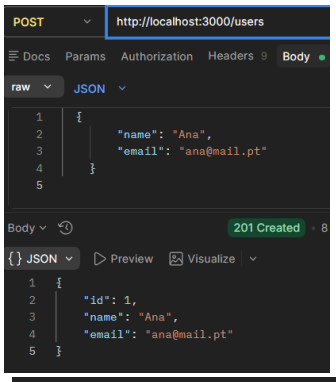
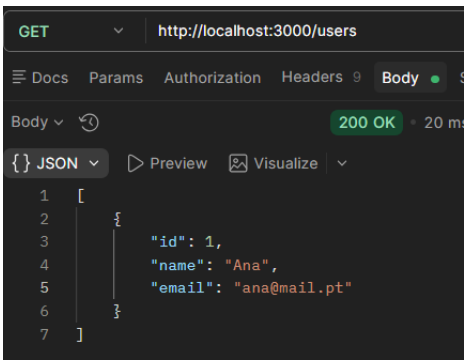
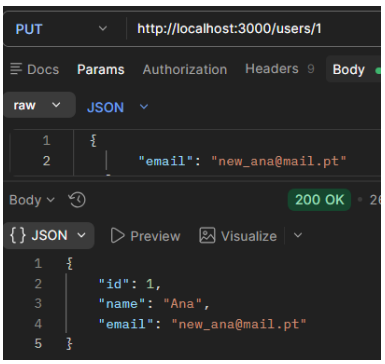
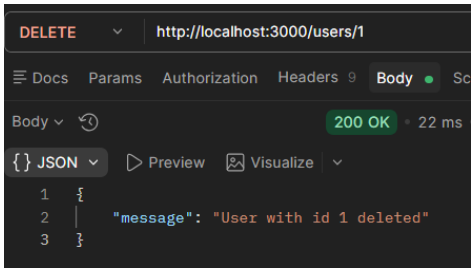
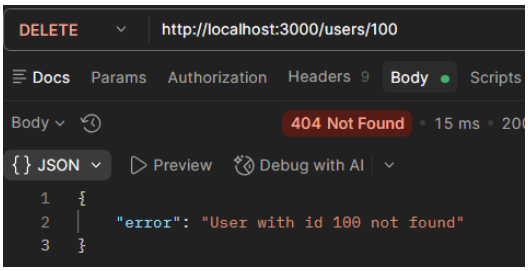
Improve your application, giving more information before the application exits, validating the input arguments and preventing file operation errors:

```
node app creates Ana
Commands:
list
create <name> <email>
update <id> <email>
delete <id>
```

```
node app create Ana
USAGE: node app.js create <name> <email>
```

```
node app create Ana ana.mail.pt
ERROR: invalid email format.
```

```
node app create Ana ana.@mail.pt
User created (id: 1)
```

```
>node app delete 2
Error: User with id 2 not found
```

8. Alter the previous `app.js` file, so that now the application is a web server that manipulates the “database” using HTTP verbs: `POST` (create), `GET` (list), `PUT` (update) and `DELETE` (delete). All required data to process a request, must be provided in the HTTP body request.